Testing and Quality

Table of Contents

Summary table

Area	Key evidence
Primary test focus	Controller / web-layer tests (WebMvcTest-based)
CI test execution	GitHub Actions workflow runs ./gradlew clean test
Build system	Gradle (build.gradle present)
Formatting / style	Spotless plugin configured in build.gradle
Test libraries declared	rest-assured, spring-boot-starter-test, spring-security-test (build.gradle)

Overview

This document summarizes verifiable testing and quality-related evidence found in the repository. The analysis is strictly limited to items present in the supplied test source files, Gradle build file, GitHub Actions workflow, and application configuration. No inferences beyond explicit evidence are made.

Key observable facts:

- Multiple test classes exist that exercise API controller behavior using Spring MVC slice testing (@WebMvcTest).
- Tests run in CI via a GitHub Actions workflow that invokes the Gradle test task.
- The Gradle build config includes test dependencies (rest-assured, spring-boot-starter-test) and a code-formatting plugin (spotless).

Technical Summary

Aspect	Value
Build system	Gradle (file: build.gradle)
Java version	sourceCompatibility = 11 (build.gradle)
CI provider and job	GitHub Actions workflow: .github/workflows/gradle.yml (job runs on ubuntu-latest)
CI test step	Step named "Test with Gradle" runs ./gradlew clean test (workflow file)

Declared test dependencies	testImplementation dependencies in build.gradle: rest-assured, json-path, xml-path, spring-mock-mvc, spring-security-test, spring-boot-starter-test, mybatis spring test
Formatting tool	Spotless plugin configured in build.gradle (com.diffplug.spotless)
Application runtime datasource (dev)	spring.datasource.url=jdbc:sqlite:dev.db (application.properties)
Controller test pattern	@WebMvcTest used in test classes (test sources)

Detected Test Strategy

Only test categories explicitly evidenced are described below.

- Web-layer (controller) slice tests:
  - Evidence: Multiple test classes annotated with @WebMvcTest (for example ArticleApiTest, ArticlesApiTest, ArticleFavoriteApiTest, CommentsApiTest).
  - These tests instantiate MockMvc via @Autowired MockMvc and exercise controller endpoints using RestAssuredMockMvc / MockMvc.
  - Tests use @Import to include WebSecurityConfig and JacksonCustomizations in the slice context.
- Mocking of dependencies in slice tests:
  - Evidence: @MockBean is used to provide mocked beans for services and repositories (for example ArticleQueryService, ArticleRepository, ArticleCommandService, ArticleFavoriteRepository, CommentRepository).
  - Tests use Mockito-style stubbing (when(...).thenReturn(...)) and verification (verify(...)).

No other test categories (for example full integration tests that exercise the real database, performance tests, contract tests, mutation tests, or explicit static-analysis quality gates) are explicitly evidenced in the supplied artifacts.

Testing Frameworks and Tooling

Only frameworks and tooling that appear in test sources or build/CI configuration are listed.

- Rest Assured (module: spring-mock-mvc)
  - Evidence: test classes import io.restassured.module.mockmvc.RestAssuredMockMvc and use given()/when() and RestAssuredMockMvc.mockMvc(mvc) patterns.
- Spring Boot test support / Spring MVC test
  - Evidence: tests use @WebMvcTest, MockMvc, @MockBean, and @Import; build.gradle includes org.springframework.boot:spring-boot-starter-test as testImplementation.
- JUnit 5 (JUnit Platform)
  - Evidence: test classes use org.junit.jupiter.api.Test and @BeforeEach; Gradle test task configured to useJUnitPlatform().
- Mockito (mocking and verification)
  - Evidence: static imports from org.mockito and use of when(...).thenReturn(...) and verify(...) in tests; spring-boot-starter-test includes Mockito.

- Spring Security test support
    - Evidence: testDependency testImplementation 'org.springframework.security:spring-security-test' in build.gradle and tests import WebSecurityConfig and use Authorization header in requests.
- Spotless (code formatting)
    - Evidence: plugin com.diffplug.spotless present in build.gradle and configured for java/googleJavaFormat.
- Gradle build and wrapper
    - Evidence: build.gradle present; CI workflow executes ./gradlew clean test.

Quality and Coverage Evidence

Explicit evidence found in the repository related to quality and coverage tooling:

- Spotless code formatting plugin:
    - Evidence: build.gradle includes the plugin com.diffplug.spotless and a java/googleJavaFormat configuration.
- CI runs tests:
    - Evidence: .github/workflows/gradle.yml contains a job that runs ./gradlew clean test.

Absent or not evidenced (explicitly verifiable):

- No JaCoCo configuration or plugin is present in build.gradle.
- No SonarQube or other static-analysis/coverage reporting plugins or steps are present in build.gradle or in the GitHub Actions workflow.
- Tests do not include any explicit test-report publishing steps in CI (the workflow shows only checkout, JDK setup, cache, and test invocation).

Note: The above list is limited to explicit references found in the provided build and CI files; it does not infer the presence of any coverage artifacts or external quality dashboards.

Representative Test Evidence

Representative classes, annotations, patterns, and concrete behaviors observed in the test sources:

- Test classes (examples)
    - ArticleApiTest.java
    - ArticleFavoriteApiTest.java
    - ArticlesApiTest.java
    - CommentsApiTest.java
- Common annotations and imports
    - @WebMvcTest(...) — controller slice configuration
    - @Import({WebSecurityConfig.class, JacksonCustomizations.class}) — explicit imports into test context
    - @MockBean — to replace beans in the test slice
    - @Autowired MockMvc — to obtain MockMvc instance
    - @BeforeEach and @Test from JUnit Jupiter
    - Use of RestAssuredMockMvc to perform requests in tests:

```
RestAssuredMockMvc.mockMvc(mvc);
given()
```

```
    .contentType("application/json")
    .header("Authorization", "Token " + token)
    .body(param)
    .when()
    .post("/articles")
    .then()
    .statusCode(200)
    .body("article.title", equalTo(title));
```

- Mocking and verification patterns
    - Stubbing service/repository behavior:

```
when(articleQueryService.findBySlug(eq(slug), eq(null)))
    .thenReturn(Optional.of(articleData));
```

    - Verifying interactions with repositories/services:

```
verify(articleRepository).remove(eq(article));
verify(articleCommandService).createArticle(any(), any());
```

- Test assertions
    - HTTP status assertions and JSON body assertions using Rest Assured style:

```
.then()
.statusCode(200)
.body("article.slug", equalTo(slug))
.body("article.body", equalTo(articleData.getBody()));
```

- Security handling in tests
    - Tests include Authorization header when exercising endpoints and import WebSecurityConfig into the test context.

Gaps and Risks

The following items are not evidenced in the provided materials; each absence is stated explicitly and the likely risk described.

- Absence of integration tests that exercise persistence or the real datasource:
    - Evidence gap: Tests shown are @WebMvcTest slice tests that use @MockBean for repositories and services; no test class in the provided snippets instantiates the full Spring context or connects to the configured SQLite datasource.
    - Risk: Potential regressions in MyBatis mappers, SQL, transaction configuration, or database migration interactions may not be caught by current slice tests.
- No explicit coverage instrumentation or reporting configured:
    - Evidence gap: build.gradle does not contain JaCoCo or other coverage plugins; GitHub Actions workflow does not publish coverage reports.
    - Risk: No automated measurement of code coverage is produced, reducing visibility into untested code areas.

- No static-analysis or quality gates in CI:
    - Evidence gap: workflow runs tests but does not invoke SonarQube, SpotBugs, PMD, or similar analysis; build.gradle does not configure such plugins.
    - Risk: Potential code-quality issues (bugs, security vulnerabilities, maintainability concerns) may not be detected automatically.
- No test-report publishing in CI:
    - Evidence gap: GitHub Actions workflow runs tests but there is no step to collect or publish JUnit or other test reports as CI artifacts.
    - Risk: Limited traceability for test failures across CI runs and reduced capability to analyze historical test results.
- No end-to-end or contract tests evidenced:
    - Evidence gap: Tests target controller slices with mocked collaborators; no tests appear to validate full request-to-database flows or external contract behavior.
    - Risk: API contract regressions or integration mismatches with clients may not be surfaced prior to deployment.
- No explicit security-scanning steps in CI:
    - Evidence gap: Although tests import WebSecurityConfig and include Authorization headers, the CI workflow does not run dedicated security scanners.
    - Risk: Security vulnerabilities in dependencies or configuration may remain undetected.

Recommendations

The following items are improvements suggested for the repository. They are recommendations only and are not claimed to be present in the current codebase.

1. Add integration tests that exercise persistence and MyBatis mappers
    - Purpose: Validate SQL mappings, MyBatis configuration, Flyway migrations, and runtime interactions with the configured datasource (sqlite:dev.db or a CI-provisioned test database).
    - Rationale: Current tests are controller slices with mocked repositories; integration tests would reduce the risk of runtime persistence issues.
2. Introduce coverage instrumentation and reporting
    - Example improvement: Enable JaCoCo in the Gradle build and publish coverage reports as CI artifacts or to a coverage dashboard.
    - Rationale: Provides visibility into tested areas and helps prioritize additional tests.
3. Add static-analysis and quality gates to CI
    - Example improvement: Configure a static analyzer (SpotBugs/PMD/Checkstyle or SonarCloud integration) within the Gradle build and add a CI step to fail the build on critical issues.
    - Rationale: Automated quality checks help maintain code health and detect issues early.
4. Publish test reports and retain artifacts in CI
    - Example improvement: Enhance the GitHub Actions workflow to upload JUnit XML and coverage reports as workflow artifacts for post-mortem analysis.
    - Rationale: Facilitates debugging of CI failures and historical trend analysis.
5. Expand test matrix to include negative authentication/authorization flows and explicit security scenarios

- Purpose: Although current tests assert some 403 outcomes, consider explicit tests for authentication failures, token expiration, and role-based access where applicable.
- Rationale: Improves confidence in security posture and reduces regressions in access control logic.

6. Consider end-to-end or contract testing for public APIs

    - Purpose: Add contract tests (consumer-driven contracts or API schema validation) or E2E tests that run against a deployed test environment.
    - Rationale: Ensures API changes do not break external consumers.

7. Leverage Spotless and add CI enforcement

    - Improvement: The project already uses Spotless; add a CI step to run spotlessCheck and fail the build on formatting violations.
    - Rationale: Enforces consistent formatting and avoids style drift.

Implementation of the above recommendations should follow project priorities and resource planning. Each recommendation is intended as an actionable improvement and not as a description of current repository capabilities.